SHOPEASE · FLUTTER SHOPPING APP

How the Features Work

For each feature: what the user sees, and how it is built.

> Files are inside `lib/`. "Controller" means a `ChangeNotifier` class provided with Provider. All product data comes from the DummyJSON API through `ProductRepository`.

Getting in

1. Splash screen

What it does: shows the animated logo and illustration, then opens Home (if already logged in) or Login.

How: `views/splash/splash_view.dart` runs a 2.4-second `AnimationController` (scale and fade for the logo, a growing progress bar). At the same time it loads the product catalogue with `Future.wait` together with a 2.6-second minimum delay, then checks `AuthController.isLoggedIn` to choose the next page.

2. Login

What it does: Email/Phone toggle, password with show/hide, Login button, error messages. A demo card fills the form with one tap; the group icon lists all demo accounts.

How: `views/auth/login_view.dart` with rules in `core/validators.dart`: e-mail format, a 10-digit mobile number starting 6 to 9, password of at least 6 characters. Live validation starts after the first tap on Login. A wrong password shows a red banner and shakes the form (an animation driven by a sine curve) with a vibration. The sliding Email/Phone toggle is an `AnimatedAlign`. The form is in a scroll view, so it fits small screens and the keyboard.

3. Accounts and separate data

What it does: three fixed logins; each account has its own cart, wishlist, orders and recent searches.

How: `DemoAccounts.authenticate()` checks e-mail (or phone) and password. On success `AuthController` calls `StorageService.useAccount(email)`, which prefixes every storage key with that e-mail, and saves the session. `main.dart` listens for the change and calls `reload()` on the cart, wishlist and search controllers so they read the new account's data. Logout clears the scope and the lists empty out. On the next launch the saved session skips Login.

Browsing

4. Home

What it does: greeting with the user's first name, search bar, filter button, three sliding banners, five category shortcuts, a Best Deals strip, sort chips and the Trending grid.

How: `views/home/home_view.dart` is one scrollable page built from slivers. The banners (`home_banner.dart`) are a `PageView` moved by a 4-second timer; "Shop Now" opens that category. Best Deals is the 10 products with the largest discount (`CatalogController.bestDeals`). The sort chips (Popular, Top Rated, Price Low, Price High, Best Discount) call `updateQuery` on the paged controller. Pull-to-refresh reloads the list.

5. Infinite scrolling

What it does: products appear 10 at a time; the next page is added automatically as the user nears the bottom.

How:

```
scroll within 500 px of the end → loadMore() → 700 ms wait → add next page → notifyListeners() → grid grows
```

- `PagedProductsController` holds all matches but exposes only one page more each time `loadMore()` runs. Flags `isLoadingMore` and `hasMore` stop duplicate or needless loads; a request counter drops results from an outdated filter.
- `PagedProductGrid` (`widgets/paged_product_grid.dart`) listens to the `ScrollController`, shows a spinner footer while loading and "You've seen it all!" at the end. It also re-checks after each rebuild, so a tall screen that is not yet full keeps loading.
- Page size: 10 (Home, categories, search), 8 (Best Sellers). The whole list is downloaded once and paged on the phone, so search, sort and filters can be combined freely. The 700 ms wait is simulated so the loader is visible.
- The same two classes power Home, Best Sellers, category pages and search results.

6. Search

What it does: results update as you type; recent and popular searches; chips for All, Men, Women, Electronics, Home, Beauty, Sports.

How: `views/search/search_view.dart` waits 350 ms after the last key (a `Timer`) before searching. `ProductQuery` scores each product: title starts with the word 100, title contains it 80, brand 60, category 40, description 20, and keeps matches ordered by score. Recent searches (up to 8) are saved per account by `RecentSearchController`. A category chip limits the search to that group's categories.

7. Filters and sorting

What it does: a bottom sheet with Sort by (Relevance, Price low to high, Price high to low, Customer rating) and a price range slider with Clear and Apply.

How: `widgets/filter_sheet.dart` returns a new `ProductQuery` (an immutable object with search text, categories, sort and price limits). The controller applies it with `ProductQuery.apply()` and restarts from page one. The slider runs $0 to $500; at $500 there is no upper limit. A red dot on the filter button shows when a sort or price filter is active.

8. Categories

What it does: six category cards with a picture and item count, a search field and group chips (All, Fashion, Electronics, Home, Beauty, Sports). A card opens that category's product list.

How: `ShopCategory` (`models/category.dart`) maps each app category to the API's category names. The count and cover picture are computed from the loaded products by `CatalogController`. The category page reuses the paged grid with a `ProductQuery` limited to those names.

Buying

9. Product details

What it does: swipeable image gallery, name, brand, rating, price with discount, stock text, a Color and Size choice, quantity, Add to Cart and a heart.

How: `views/product/product_detail_view.dart`. The card image and the gallery share a `Hero` tag, so the picture animates between pages. The API provides no colour or size data (its extra images are only more views of the same item), so every product shows one default choice, Color "Default" and Size "One size", rather than made-up options. Add to Cart and the quantity bar sit in the page's bottom slot, so pop-up messages appear above them. Out-of-stock products disable the button.

10. Cart

What it does: cart lines with quantity +/−, delete (swipe or bin) with Undo, clear-all with confirmation, and an order summary: subtotal, shipping and total.

How: `CartController` keeps the lines. There is one line per product, so adding the same product again raises its quantity. Removing returns the line's position so Undo can put it back in place. Shipping is free from a $50 subtotal, otherwise $4.99. Every change is saved through `StorageService` and the tab badge updates through Provider.

11. Checkout and orders

What it does: Checkout shows a loading button for about a second, then an animated success page with the order id, item count and total. My Orders (Profile) lists past orders.

How: no payment is made. `CartController.placeOrder()` creates an `Order` (id like SE123456), saves it with the account's orders and empties the cart. The success page is `views/cart/order_success_view.dart`; the list is `views/profile/orders_view.dart`. The status label "Processing" is fixed text.

12. Wishlist

What it does: the heart on any product saves it; the Wishlist tab shows the saved items.

How: `WishlistController` stores only product ids per account. `WishlistButton` uses `context.select`, so only the tapped heart rebuilds, with a small bounce animation. The Wishlist tab looks the ids up in the catalogue. An empty list shows a message with a shortcut back to shopping.

App-wide

13. Profile

What it does: avatar in the account's colour, name and e-mail; My Orders, Wishlist and Settings work; Log out asks for confirmation. Addresses, Payment Methods, Notifications and Help & Support only show a "not part of this demo" message.

How: `views/profile/profile_view.dart` reads the user from `AuthController`. Log out clears the session and returns to Login.

14. Light, Dark and System theme

What it does: Settings → Appearance switches the whole app instantly and remembers the choice. System follows the phone's setting.

How: `ThemeController` saves the mode. `AppColors` returns light or dark colours according to one flag. When the mode changes, `main.dart` flips the flag, builds a new Material theme and marks every open widget for rebuild, so colours update without losing the current page, scroll position or typed text. The status bar and navigation bar icons change too.

15. Navigation, loading and responsiveness

- **Bottom navigation:** five tabs kept alive with an `IndexedStack`, so each tab keeps its scroll position. Back on another tab returns to Home first (`PopScope`).
- **Loading and errors:** shimmer placeholders while products load; a retry message on failure; "No products found" for empty results; an "Offline" note when placeholder products are used.
- **Micro-interactions:** press-scale on taps (`Tappable`), fade-in of new cards, animated toggle, hearts and quantity numbers.
- **Responsive:** grid columns = available width ÷ 185, limited to 2 to 5; card height follows the text size setting; wide screens get larger page padding.